

Dependency levels in Angular



Root Level

Platform Level

Module Level

Component Level

Directive Level



Common Mistake

Most devs do this:

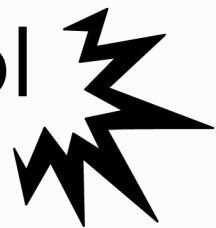


`providedIn: 'root'`



...Even when they shouldn't.

It works... until your lazy modules go out of control



Root-Level Injection

Root Level (providedIn: 'root')

- ✓ Singleton for the entire app
- 📦 Best for global services like API wrappers, auth, or state managers

```
@Injectable({ providedIn: 'root' })
```

💡 Easy to use, but use wisely — overuse can cause tight coupling.

Module-Level Injection

Module Level (Scoped to an NgModule)

- ✓ Separate instance per module
- ⚡ Especially powerful with lazy-loaded modules

```
@NgModule({  
  providers: [FeatureService]  
})
```

💡 Common mistake: assuming it's shared. It's NOT across modules!

Component-Level Injection

Component Level

✓ Scoped to the component and its view children

🎯 Best for UI-bound logic like dropdown state or timers

```
@Component({  
  providers: [LocalStateService]  
})
```

💡 Every instance of the component gets its own copy of the service

Directive-Level Injection

Directive Level (Scoped to Element Usage)

✓ Service instance is tied to the element where directive is applied

🔧 Ideal for reusable behavior — like drag/drop, tooltips, validation

```
@Directive({  
  selector: '[customTooltip]',  
  providers: [TooltipService]  
})
```

💡 Highly reusable but underutilized — great for clean DOM logic.

Platform-Level Injection

Platform Level (Rare but Powerful)

✓ Lives as long as the Angular platform itself

↻ Useful for multi-app environments, shared services across bootstrapped apps

```
platformBrowserDynamic().bootstrapModule(AppModule)
```

⚠ Usually handled by Angular core — not used in typical apps

Scope matters



Slide 07

Level	Scope	Example
Root	Whole app	providedIn: 'root'
Module	Module-only	@NgModule.providers
Component	Component tree	@Component.providers
Directive	Usage-specific	@Directive.providers
Platform	All apps	bootstrapModule()



The Problem It Solves

Before knowing this:

- 😵 Unexpected behavior
- 🐛 Duplicate service bugs
- 🧠 Confusing lazy modules

After knowing this:

- ✅ Predictable DI
- ✅ Better app performance
- ✅ Cleaner architecture

